

05 — Stacks & Queues

DSA in Python — Interview Prep

Stacks & Queues — Tutorial

Covers the 10 problems solved in this folder. Stacks and queues are the two simplest possible restrictions on where you can add/remove elements (LIFO vs FIFO) — most of this topic is about recognizing *which* restriction a problem secretly needs, since neither structure is built into Python as a distinct type (a `list` serves as a stack; `collections.deque` serves as a queue or a double-ended structure).

The patterns

1. Plain LIFO matching (01)

The most direct use of a stack: nested structure where the most recently opened thing must be the next one closed. Push on open, pop-and-compare on close, and a non-empty stack at the end means something was left unclosed. This exact "push open, resolve on close" shape reappears in `10_decode_string` for a harder version of the same idea.

2. Auxiliary stack for $O(1)$ extra queries (02)

When a stack needs to answer a question that isn't naturally $O(1)$ (like "what's the minimum right now?"), keep a second stack in lockstep that caches the answer to that question *as of each push*. `min_stack` computes `min(new_val, previous_min)` on every push and pops in lockstep with the main stack, so `get_min()` never has to re-scan.

3. Postfix expression evaluation (03)

Postfix (RPN) notation is designed around a stack: by the time an operator token is read, both its operands are already sitting on top of the stack in the right order, so evaluation is just repeated pop-pop-compute-push. No precedence or parenthesis handling needed — that's the entire reason RPN exists as a notation.

4. Monotonic stack (04, 05, 06)

The single most important pattern in this folder. Keep a stack whose values are always increasing or always decreasing (top to bottom). When a new value would break that ordering, pop everything that it resolves — each popped element has just found its "next greater" (or next smaller, or right boundary) relative to the current position. Every element is pushed once and popped at most once, so despite the inner `while` loop this is $O(n)$ total, not $O(n^2)$ — same amortized argument as topic 03's shrinking sliding window.

- `daily_temperatures`: decreasing stack of indices; a warmer day resolves every colder day still stacked, recording the day-gap.
- `next_greater_element`: identical mechanism, but the pass builds a reusable `value -> next_greater` dict over all of `nums2` once, since only a subset of values (`nums1`) actually needs answering.

- `largest_rectangle_in_histogram`: increasing stack of indices. A shorter bar resolves every taller bar still stacked — each resolved bar's rectangle spans from the (now-exposed) new stack top to the current position. This is the hardest problem in the folder; the monotonic-stack idea is identical to the other two, just computing an area instead of a gap or a value.

5. Simulating one structure with the other (07, 08)

Mirror-image design problems: build a FIFO queue out of LIFO stacks, or a LIFO stack out of a FIFO queue.

- `implement_queue_using_stacks`: two stacks, `in_stack` for pushes and `out_stack` for pops. Only reverse `in_stack` into `out_stack` when `out_stack` is empty ("lazy transfer") — each element crosses from one stack to the other at most once in its lifetime, keeping amortized cost $O(1)$ despite an occasional $O(n)$ reversal.
- `implement_stack_using_queues`: a single deque, rotated by `size - 1` after every push so the newest element ends up at the front — trading push's cost ($O(n)$ rotation) for making pop/top trivial ($O(1)$ front access), the opposite trade-off from the stacks-based queue above.

6. Monotonic deque for a sliding window (09)

`sliding_window_maximum` combines the monotonic-stack idea from pattern 4 with topic 03's sliding window: it needs removal from *both* ends — the back (discard values that can never be the max again, same rule as pattern 4) and the front (discard indices that fell outside the window). That two-ended removal requirement is exactly why a `deque` is needed here instead of a plain stack.

7. Stack-based nested parsing (10)

`decode_string` generalizes pattern 1: instead of just matching open/close, each stack frame carries *state* (the partial string built so far, plus the repeat count) that gets paused on `[` and resumed/combined on `]`. This push-state-before-descending, pop-and-combine-on-return shape is the general template for parsing any nested grammar (JSON, arithmetic expressions with parens, HTML/ XML) with a stack instead of recursion.

Complexity summary

#	Problem	Time	Extra space
01	<code>valid_parentheses</code>	$O(n)$	$O(n)$
02	<code>min_stack</code>	$O(1)$ per op	$O(n)$
03	<code>evaluate_reverse_polish_notation</code>	$O(n)$	$O(n)$
04	<code>daily_temperatures</code>	$O(n)$	$O(n)$
05	<code>next_greater_element</code>	$O(n+m)$	$O(n)$
06	<code>largest_rectangle_in_histogram</code>	$O(n)$	$O(n)$
07	<code>implement_queue_using_stacks</code>	$O(1)$ amortized per op	$O(n)$
08	<code>implement_stack_using_queues</code>	$O(n)$ push, $O(1)$ pop/top	$O(n)$
09	<code>sliding_window_maximum</code>	$O(n)$	$O(k)$
10	<code>decode_string</code>	$O(n \cdot \text{max repeat depth})$	$O(n)$

Every monotonic-stack problem here (04, 05, 06, 09) looks like it should be $O(n^2)$ from the nested loop, but isn't — always be ready to give the amortized argument (each element pushed once, popped at most once) rather

than just asserting "it's $O(n)$."

Common interview follow-ups to be ready for

- `valid_parentheses`: "what about other paired delimiters, like custom tags?" — the `pairs` dict generalizes to any closing->opening mapping without changing the algorithm shape.
- `min_stack`: "can you do it with $O(1)$ *extra* space instead of a second full stack?" — yes, by storing the *difference* from the running min instead of the min itself, but it's a harder trick; know the auxiliary-stack version cold first.
- `next_greater_element`: "what if `nums2` could have duplicates?" — problem guarantees distinct values here; discuss what breaks (the dict key collision) if that guarantee were dropped.
- `largest_rectangle_in_histogram`: "how does this extend to 'maximal rectangle in a binary matrix'?" (LC85) — run this exact algorithm once per row, treating each row as a histogram built from consecutive 1s above it.
- `implement_queue_using_stacks`: "is push or pop the expensive operation?" — neither, in the amortized sense; be ready to explain why a single push can still trigger an $O(n)$ transfer without breaking the $O(1)$ amortized claim.
- `sliding_window_maximum`: "what about sliding window *minimum*?" — flip the monotonic direction of the deque; same mechanism.
- `decode_string`: "what if the encoding also supported letters mixed with the repeat count directly, like real-world compressed formats?" — the state-pushing template still applies, just with a richer per-frame state.

Self-check before moving on

You should be able to, without looking at the code:

- Explain the amortized $O(n)$ argument for any of the monotonic-stack problems (why the inner while loop doesn't make it $O(n^2)$).
- Write the "lazy transfer" queue-from-two-stacks from memory and explain why it's $O(1)$ amortized, not $O(n)$ per operation.
- Explain why `sliding_window_maximum` needs a deque (both-end removal) instead of a plain stack.
- Trace `decode_string` by hand on `"3[a2[c]]"`, writing out the stack's contents after each `[` and `]`.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `06_binary_search`.

Programs

01. Valid Parentheses

01_valid_parentheses.py

Explanation

Problem: Given a string containing just '(', ')', '{', '}', '[', ']', determine if the brackets are validly matched and nested.

Source : LeetCode 20 - Valid Parentheses

Example:

Input: "{[]}"

Output: True

Idea: the defining property of a stack (last-in-first-out) matches exactly how nested brackets must close - the most recently opened bracket must be the next one closed. Push opening brackets; on a closing bracket, it must match whatever is on top of the stack right now, or the string is invalid. A leftover non-empty stack at the end means something was opened but never closed.

Code

```
def is_valid(s: str) -> bool:
    stack = []
    pairs = {"(": ")", "[": "]", "{": "}"}

    for ch in s:
        if ch in pairs:
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
        else:
            stack.append(ch)

    return not stack

if __name__ == "__main__":
    tests = [
        ("()", True),
        ("()[]{}", True),
        ("]", False),
        ("[]]", False),
        ("{}", True),
        ("", True),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_valid(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Min Stack

02_min_stack.py

Explanation

Problem: Design a stack that supports push, pop, top, and retrieving the minimum element, all in $O(1)$ time.

Source : LeetCode 155 - Min Stack

Example:

```
stack = MinStack()
stack.push(-2); stack.push(0); stack.push(-3)
stack.get_min() -> -3
stack.pop()
stack.top() -> 0
stack.get_min() -> -2
```

Idea: a single stack can't answer `get_min()` in $O(1)$ after pops, since the minimum could change with every pop and re-scanning is $O(n)$.
Fix: keep a second, parallel stack where `min_stack[i]` always holds "the minimum of the main stack's bottom $i+1$ elements." Every push computes and stores that running minimum; every pop discards from both stacks together, so `min_stack[-1]` is always instantly correct.

Code

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        current_min = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(current_min)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def get_min(self) -> int:
        return self.min_stack[-1]

if __name__ == "__main__":
    stack = MinStack()
    ops = [
        (stack.push, (-2, ), None),
        (stack.push, (0, ), None),
        (stack.push, (-3, ), None),
        (stack.get_min, (), -3),
        (stack.pop, (), None),
        (stack.top, (), 0),
        (stack.get_min, (), -2),
    ]
```

```

for i, (fn, args, expected) in enumerate(ops, 1):
    got = fn(*args)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```

03. Evaluate Reverse Polish Notation

03_evaluate_reverse_polish_notation.py

Explanation

Problem: Evaluate an arithmetic expression given in Reverse Polish (postfix) Notation. Tokens are integers or one of + - * /.

Source : LeetCode 150 - Evaluate Reverse Polish Notation

Example:

Input: ["2", "1", "+", "3", "*"]

Output: 9 ((2 + 1) * 3)

Idea: postfix notation is exactly what a stack-based evaluator is for - by the time an operator is read, its two operands are already on top of the stack (in the order they were pushed), so applying the operator is just: pop twice, compute, push the result back. No parentheses or precedence rules are ever needed, which is the whole appeal of postfix.

Code

```

def eval_rpn(tokens: list[str]) -> int:
    stack: list[int] = []
    operators = {"+", "-", "*", "/"}

    for token in tokens:
        if token in operators:
            b = stack.pop()
            a = stack.pop()
            if token == "+":
                stack.append(a + b)
            elif token == "-":
                stack.append(a - b)
            elif token == "*":
                stack.append(a * b)
            else:
                stack.append(int(a / b)) # truncate toward zero, not floor
        else:
            stack.append(int(token))

    return stack[0]

if __name__ == "__main__":
    tests = [
        (["2", "1", "+", "3", "*"], 9),
        (["4", "13", "5", "/", "+"], 6),
        (["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"], 22),
    ]

    for i, (tokens, expected) in enumerate(tests, 1):
        got = eval_rpn(tokens)

```

```
status = "PASS" if got == expected else "FAIL"
print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Daily Temperatures

04_daily_temperatures.py

Explanation

Problem: Given a list of daily temperatures, return a list where answer[i] is the number of days until a warmer temperature. If there is none, answer[i] = 0.

Source : LeetCode 739 - Daily Temperatures

Example:

Input: [73, 74, 75, 71, 69, 72, 76, 73]

Output: [1, 1, 4, 2, 1, 1, 0, 0]

Idea: monotonic (decreasing) stack of *indices*. Walk left to right; whenever the current temperature is warmer than the temperature at the index on top of the stack, that's the answer for that stacked index (found its "next warmer day"), so pop it and record the gap. Keep popping while the new temperature beats the stack top - a single element only ever gets pushed once and popped once, so the whole pass is O(n) even though it looks like nested loops.

Code

```
def daily_temperatures(temperatures: list[int]) -> list[int]:
    n = len(temperatures)
    result = [0] * n
    stack: list[int] = [] # indices, temperatures decreasing bottom to top

    for i, t in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < t:
            j = stack.pop()
            result[j] = i - j
        stack.append(i)

    return result

if __name__ == "__main__":
    tests = [
        ([73, 74, 75, 71, 69, 72, 76, 73], [1, 1, 4, 2, 1, 1, 0, 0]),
        ([30, 40, 50, 60], [1, 1, 1, 0]),
        ([30, 60, 90], [1, 1, 0]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = daily_temperatures(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Next Greater Element

05_next_greater_element.py

Explanation

Problem: nums1 is a subset of nums2 (all elements distinct). For each element in nums1, find its next greater element to the right in nums2, or -1 if none exists.

Source : LeetCode 496 - Next Greater Element I

Example:

Input: nums1 = [4, 1, 2], nums2 = [1, 3, 4, 2]

Output: [-1, 3, -1]

Idea: same monotonic-stack shape as 04_daily_temperatures, but here the "gap" being recorded is the *value* at the resolving index rather than the distance to it, and the answer only needs to be looked up for the nums1 subset afterward - so the stack pass builds a value -> next_greater dict over all of nums2 once, then nums1 just does O(1) dict lookups.

Code

```
def next_greater_element(nums1: list[int], nums2: list[int]) -> list[int]:
    next_greater: dict[int, int] = {}
    stack: list[int] = [] # values, decreasing bottom to top

    for n in nums2:
        while stack and stack[-1] < n:
            next_greater[stack.pop()] = n
        stack.append(n)

    return [next_greater.get(n, -1) for n in nums1]

if __name__ == "__main__":
    tests = [
        ([4, 1, 2], [1, 3, 4, 2], [-1, 3, -1]),
        ([2, 4], [1, 2, 3, 4], [3, -1]),
    ]

    for i, (nums1, nums2, expected) in enumerate(tests, 1):
        got = next_greater_element(nums1, nums2)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Largest Rectangle In Histogram

06_largest_rectangle_in_histogram.py

Explanation

Problem: Given an array of bar heights forming a histogram (each bar width 1), find the area of the largest rectangle that fits entirely within the histogram's outline.

Source : LeetCode 84 - Largest Rectangle in Histogram

Example:

Input: [2, 1, 5, 6, 2, 3]

Output: 10 (rectangle of height 5 spanning bars at indices 2-3

gives $5 \times 2 = 10$; height 2 spanning indices 2-5 also

gives $2 \times 4 = 8$; the height-5/6 pair wins at 10)

Idea: monotonic (increasing) stack of indices. For any bar, the widest rectangle *of that bar's height* extends left and right until hitting a shorter bar - so the moment a shorter bar is encountered while scanning left to right, every taller bar still on the stack has just found its right boundary. Pop it, compute its area using the now-shorter stack top as its left boundary (or index 0 if the stack is empty, meaning nothing shorter exists to its left). A sentinel height of 0 appended at the end forces every remaining bar on the stack to be resolved by the time the scan finishes.

Code

```
def largest_rectangle_area(heights: list[int]) -> int:
    heights = heights + [0]
    stack: list[int] = [] # indices, heights increasing bottom to top
    max_area = 0

    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

    return max_area

if __name__ == "__main__":
    tests = [
        ([2, 1, 5, 6, 2, 3], 10),
        ([2, 4], 4),
        ([1], 1),
        ([], 0),
        ([5, 5, 5], 15),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = largest_rectangle_area(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Implement Queue Using Stacks

07_implement_queue_using_stacks.py

Explanation

Problem: Implement a FIFO queue using only two stacks (LIFO).

Support push, pop, peek, and empty.

Source : LeetCode 232 - Implement Queue using Stacks

Example:

```
q = MyQueue()
q.push(1); q.push(2)
q.peek() -> 1
q.pop() -> 1
q.empty() -> False
```

Idea: two stacks, `in_stack` for incoming pushes and `out_stack` for outgoing pops. Pushing is always cheap (just append to `in_stack`). Popping/peeking needs the *oldest* element, which is at the *bottom* of `in_stack` - reversing `in_stack` once into `out_stack` (via repeated pop/push) puts it on top of `out_stack`. Only refill `out_stack` when it's empty; otherwise its existing order is still correct. This "lazy transfer" is what keeps amortized cost $O(1)$ per operation - each element is moved from `in_stack` to `out_stack` at most once in its lifetime.

Code

```
class MyQueue:
    def __init__(self):
        self.in_stack: list[int] = []
        self.out_stack: list[int] = []

    def push(self, x: int) -> None:
        self.in_stack.append(x)

    def _transfer_if_needed(self) -> None:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())

    def pop(self) -> int:
        self._transfer_if_needed()
        return self.out_stack.pop()

    def peek(self) -> int:
        self._transfer_if_needed()
        return self.out_stack[-1]

    def empty(self) -> bool:
        return not self.in_stack and not self.out_stack

if __name__ == "__main__":
    q = MyQueue()
    ops = [
        (q.push, (1,), None),
        (q.push, (2,), None),
        (q.peek, (), 1),
        (q.pop, (), 1),
        (q.empty, (), False),
        (q.push, (3,), None),
        (q.push, (4,), None),
        (q.pop, (), 2),
        (q.pop, (), 3),
        (q.pop, (), 4),
        (q.empty, (), True),
    ]
```

```

for i, (fn, args, expected) in enumerate(ops, 1):
    got = fn(*args)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```

08. Implement Stack Using Queues

08_implement_stack_using_queues.py

Explanation

Problem: Implement a LIFO stack using only a queue (or queues).

Support push, pop, top, and empty.

Source : LeetCode 225 - Implement Stack using Queues

Example:

```

s = MyStack()
s.push(1); s.push(2)
s.top() -> 2
s.pop() -> 2

```

Idea: the mirror image of 07's problem. A single deque is enough: after pushing a new element, rotate the queue by (size - 1) steps so the just-pushed element moves to the *front*. That makes the queue's front always equal to the most-recently-pushed element - i.e. LIFO order - so pop/top just operate on the front like a normal queue operation, no second data structure needed.

Code

```

from collections import deque

class MyStack:
    def __init__(self):
        self.q: deque = deque()

    def push(self, x: int) -> None:
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:
        return self.q.popleft()

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return not self.q

if __name__ == "__main__":
    s = MyStack()
    ops = [
        (s.push, (1, ), None),
        (s.push, (2, ), None),
        (s.top, (), 2),

```

```

        (s.pop, (), 2),
        (s.empty, (), False),
        (s.push, (3,), None),
        (s.push, (4,), None),
        (s.pop, (), 4),
        (s.pop, (), 3),
        (s.pop, (), 1),
        (s.empty, (), True),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```

09. Sliding Window Maximum

09_sliding_window_maximum.py

Explanation

Problem: Given an array `nums` and a window size `k`, return the maximum value in each sliding window of size `k` as it moves left to right.

Source : LeetCode 239 - Sliding Window Maximum

Example:

Input: `nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`

Output: `[3, 3, 5, 5, 6, 7]`

Idea: monotonic (decreasing) deque of **indices**. Before adding the current index, pop any indices from the back whose values are smaller than the current value - they can never be the max of any future window while a bigger, more-recent value is also in play, so they're permanently useless and safe to discard. The front of the deque is always the index of the current window's maximum; if that index has fallen outside the window (`dq[0] <= i - k`), pop it from the front. Combines the monotonic-stack idea from 04/05/06 with a deque instead of a stack, because elements need to be removed from **both** ends (back for the "useless smaller values" rule, front for the "fell out of the window" rule).

Code

```

from collections import deque

def max_sliding_window(nums: list[int], k: int) -> list[int]:
    dq: deque = deque() # indices, values decreasing front to back
    result = []

    for i, n in enumerate(nums):
        while dq and nums[dq[-1]] < n:
            dq.pop()
        dq.append(i)

        if dq[0] <= i - k:
            dq.popleft()

        result.append(nums[dq[0]])
    return result

```

```

        if i >= k - 1:
            result.append(nums[dq[0]])

    return result

if __name__ == "__main__":
    tests = [
        ([1, 3, -1, -3, 5, 3, 6, 7], 3, [3, 3, 5, 5, 6, 7]),
        ([1], 1, [1]),
        ([9, 11], 2, [11]),
        ([4, -2], 2, [4]),
    ]

    for i, (nums, k, expected) in enumerate(tests, 1):
        got = max_sliding_window(nums, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

10. Decode String

10_decode_string.py

Explanation

Problem: Given an encoded string like "3[a2[c]]", decode it. The encoding rule is k[encoded_string], meaning encoded_string is repeated k times. Nesting is allowed.

Source : LeetCode 394 - Decode String

Example:

Input: "3[a2[c]]"

Output: "accaccacc"

Idea: a stack of (string-so-far, repeat-count) pairs, one entry per level of nesting. On '[', the current partial string and the number built up before it are pushed - "paused" - and a fresh partial string starts for the nested level. On ']', the nested level is complete: pop the paused (outer_string, count), and the finished nested string is repeated **count** times and appended onto outer_string, which becomes the new "current" string. This is the general pattern for any nested-structure parsing problem: push state before descending into a nested level, pop and combine when that level closes.

Code

```

def decode_string(s: str) -> str:
    stack: list[tuple[str, int]] = []
    current_str = ""
    current_num = 0

    for ch in s:
        if ch.isdigit():
            current_num = current_num * 10 + int(ch)
        elif ch == "[":
            stack.append((current_str, current_num))
            current_str = ""
            current_num = 0

```

```

        elif ch == "]":
            prev_str, num = stack.pop()
            current_str = prev_str + current_str * num
        else:
            current_str += ch

    return current_str

if __name__ == "__main__":
    tests = [
        ("3[a]2[bc]", "aaabcbc"),
        ("3[a2[c]]", "accaccacc"),
        ("2[abc]3[cd]ef", "abcabccdcdef"),
        ("abc3[cd]xyz", "abccdcxyz"),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = decode_string(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got!r}, expected {expected!r})")

```